

QUESTION 2:

Aim :

To solve the recurrence relation using the substitution method and determine its time complexity.

```
def index_builder(n):  
    if n <= 1:  
        return 1  
    return index_builder(n // 2) + 1  
n = int(input("Enter number of web pages: "))  
print("Operations Required:", index_builder(n))
```

```
Enter number of web pages: 32  
Operations Required: 6
```

Step-by-Step Expansion:

Given:

$$T(n) = T(n/2) + 1$$

$$\text{Expand once} = T(n/4) + 1 + 1$$

$$\text{Expand again} = T(n/8) + 1 + 1 + 1$$

$$\text{After } k \text{ expansions: } T(n) = T(n/2^k) + k$$

$$\text{Stop when } n/2^k = 1$$

$$\text{Therefore : } 2^k = n$$

Taking logarithm , $k=\log_2 n$

Substitute, $T(n)=T(1)+\log_2 n$

Since, $T(1)=1$

Final result , $T(n)=1+\log_2 n$

Ignoring constants , $T(n)=\Theta(\log n)$

Complexity:

Time Complexity : $\Theta(\log n)$

Space Complexity : $\Theta(\log n)$

Scalability Analysis:

Number of Pages (n)	Operations
8	4
16	5
32	6
64	7
128	8
256	9

Graph Values

n	$\log_2(n)$
8	3
16	4
32	5
64	6
128	7

Interpretation:

The recurrence shows that the work increases logarithmically with the input size. Even if the number of web pages doubles, only one additional recursive step is required. This demonstrates excellent scalability for building a search engine index.

Conclusion:

The recurrence relation , $T(n)=T(n/2)+1$ solves to,
 $T(n)=\Theta(\log_{[f_0]}n)$. This logarithmic complexity makes the algorithm highly efficient for large datasets because the number of operations grows very slowly as the input size increases.